

# **Embedded System & IoT**

## **Syllabus**

Introduction to Embedded and IOT Systems: Definition, Examples and components of embedded Systems, Classification of an Embedded system. Architecture of Embedded system. General purpose computers vs embedded system, Embedded System Design Process, Various Embedded cores controller. Embedded system with IOT connectivity.

Hardware/Software Co-design for Embedded Systems: Microcontrollers for embedded systems, 32-bit RISC Architectures for embedded Design, ARM architectural details, The ARM programmer's model, ARM development tools, ARM microcontroller programming in C, Peripheral Interfacing with ARM, Basic Wire and Wireless Protocols like, UART, I2C, SPI, PLCC, Bluetooth, WiFi, Zig-Bee and LoRa for IoT applications.

Embedded Operating Systems: Operating system requirements for Embedded systems, Fundamentals of Real Time Operating System (RTOS), Operating system services, Process, Task and Thread, System calls, Timer and Event Function, Memory management, File and I/O subsystem management, Device Management, Device drivers and It's Programming for Embedded platform

OS based Software development: Programming in higher level languages on embedded OS platform, Communication protocols and it's applications , Embedded Systems with Internet of Things (IoT) and Cloud support

Introduction to IOT based Embedded Systems: Basic architecture of an IoT based Embedded Systems., Embedded Hardware for IoT applications, like Raspberry Pi, Arduino, and ARM development board, IoT Cloud Platform and IoT client applications on mobile phones.

Case Studies of Embedded Systems: Embedded application development through ARM based development boards, Development of mini Project on new version of Operating systems and development board. That project should also address to the current societal needs.

## **Books**

1. Internet of Things – A Hands-on Approach, Arshdeep Bahga and Vijay Madisetti, Universities Press, 2015, ISBN: 9788173719547
2. Getting Started with Raspberry Pi, Matt Richardson & Shawn Wallace, O'Reilly (SPD), 2014, ISBN: 9789350239759
3. Jan Holler, Vlasios Tsiatsis, Catherine Mulligan, Stefan Avesand, Stamatis Karnouskos, David Boyle, "From Machine-to-Machine to the Internet of Things: Introduction to a New Age of Intelligence", 1st Edition, Academic Press, 2014.
4. Designing Embedded Systems and the Internet of Things (IoT) with the ARM mbed (Wiley - IEEE)

# Unit-1

## 1. Embedded System Introduction

We live in an era where pervasive (spreading) computing exists everywhere, right from a small handheld device such as a mobile phone to the electronic control units within automobiles or avionics. Today, large volumes of information is getting processed and communicated over the Internet every microsecond. Buzz words such as Cloud Computing, Big Data Mining and Internet of Things are everywhere.

There are two broad classifications of computing systems –

- general purpose computing system and
- embedded computing systems.

If we define these in simple words, general purpose computing systems are those used in desktop or laptop computers, which can process several different applications. An embedded system refers to any device that has some computational intelligence in it. It is generally used as a standalone system that repeatedly performs a specific task or as part of a large system to perform multiple tasks with the required hardware and software embedded within. Systems used in printers, washing machines, mp3 players; CT scan machines etc. are great examples of embedded systems.

## 2. Components of embedded Systems

The **components of an embedded system** can be broadly divided into **hardware**, **software**, and sometimes **firmware**. These components work together to perform specific, dedicated functions.

### Hardware Components

These are the physical parts of the embedded system.

| Component                                           | Description                                                                                                                                                     |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Processor / Microcontroller / Microprocessor</b> | The brain of the system. Executes instructions. Microcontrollers (like STM32, ATmega) are more common in embedded systems than general-purpose microprocessors. |
| <b>Memory (RAM and ROM)</b>                         | - <b>RAM</b> : Temporary data storage for execution (volatile)- <b>ROM / Flash</b> : Stores the program code and permanent data (non-volatile)                  |
| <b>Input Devices</b>                                | Sensors, buttons, keyboards, or any component that sends data to the system                                                                                     |
| <b>Output Devices</b>                               | Displays, LEDs, motors, buzzers — devices that present or act on processed data                                                                                 |

|                                 |                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Timers/Counters</b>          | Used for generating delays, counting events, or scheduling tasks                              |
| <b>Communication Interfaces</b> | For data exchange with external systems (e.g., UART, I2C, SPI, CAN, Ethernet, USB, etc.)      |
| <b>Power Supply</b>             | Provides regulated voltage and current to the system                                          |
| <b>A/D and D/A Converters</b>   | Converts analog signals to digital (ADC) and vice versa (DAC), crucial for sensor interfacing |

## Software Components

These are the programs that run on the embedded hardware to perform tasks.

| <b>Component</b>                   | <b>Description</b>                                                                                                                                  |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Application Code</b>            | The logic that controls the device's function (e.g., temperature control, motor speed regulation)                                                   |
| <b>Operating System (Optional)</b> | - <b>RTOS (Real-Time Operating System)</b> is often used in complex systems- Simple systems may run without an OS ( <b>bare metal programming</b> ) |
| <b>Device Drivers</b>              | Interface software between the hardware and application (e.g., sensor drivers, display drivers)                                                     |
| <b>Middleware</b>                  | Software that provides services beyond drivers, like communication protocols (TCP/IP stack, CAN libraries)                                          |
| <b>Bootloader</b>                  | A small program that starts first and helps load the main application code into memory                                                              |

## 3. Firmware

- A hybrid of software and hardware.
- Stored in ROM/Flash and directly controls hardware.
- Usually refers to **low-level control programs**.

## 4. Other Components (Optional but Common)

| <b>Component</b>               | <b>Description</b>                                                   |
|--------------------------------|----------------------------------------------------------------------|
| <b>Watchdog Timer</b>          | Resets the system if it crashes or freezes                           |
| <b>Real-Time Clock (RTC)</b>   | Keeps track of actual time (used in clocks, logs, alarms)            |
| <b>Sensors &amp; Actuators</b> | Input from real-world data (sensors) and physical output (actuators) |
| <b>External Storage</b>        | SD card, EEPROM, etc., for logging or configuration files            |

## 3. Host and Target concept:

### What is a Host Machine?

The **host machine** is the **development system** — usually your **PC or workstation** — where:

- Code is written
- Compiled
- Debugged
- Simulated (sometimes)

#### ❓ **Common tools on the host:**

- Text editor/IDE (e.g., Eclipse, VS Code)
- Cross-compiler (e.g., `arm-none-eabi-gcc`)
- Debugger (e.g., GDB)
- Emulator or simulator (optional)

## **What is a Target Machine?**

The **target machine** is the **actual embedded hardware** where the compiled code is finally executed. This could be:

- A microcontroller board (e.g., STM32, Arduino)
- A System on Chip (SoC) like Raspberry Pi
- A custom hardware board

| Feature          | Host Machine                    | Target Machine                      |
|------------------|---------------------------------|-------------------------------------|
| Role             | Development & Testing           | Execution                           |
| Architecture     | General-purpose CPU (e.g., x86) | Embedded processor (e.g., ARM, AVR) |
| Tools Used       | Compiler, Debugger, Simulator   | Loader, OS (RTOS or bare metal)     |
| Operating System | Windows, Linux, macOS           | RTOS, Linux (sometimes), or None    |
| Communication    | USB, Serial, JTAG, Ethernet     | Receives code via same interfaces   |

## **4. Embedded Systems Applications:**

- i. Avionics: Navigation systems, Inertial Guidance Systems, GPS Receivers etc
- ii. Automotives: Automatic Breaking System, Air Bag System, Ignition Systems etc.
- iii. Consumer Electronics: MP3 Players, Mobile Phones, Video Game Consoles, Digital Cameras, Set top Boxes, Camcorders etc.
- iv. Telecommunication: Cell Phones, Telephone Switches etc.
- v. Medical Systems: ECG, EEG, MRI, CT scan, BP Monitors, etc.
- vi. Military Applications
- vii. Industries
- viii. Home Appliances: AC, TV, DVD Players, Washing machine, Refrigerators, Microwave Oven, etc.
- ix. Computer Peripherals: Printers, Scanners, Fax Machines.
- x. Computer Networking Systems: Network Routers, Switches, Hubs etc.
- xi. Security Systems: Intruder Detection Alarms, CC Cameras, Fire Alarms
- xii. Measurement & Instrumentation: Digital Multimeters, Digital CRO's, Logic Analyzers, PLC Systems.
- xiii. Banking and Retail: ATM, Currency Counters. — Card readers: Barcode, Smart Card Readers, Hand Held Devices.

## 5. Classification of an Embedded system

Embedded systems can be classified in several ways, depending on **functionality, performance, complexity, real-time requirements, and application domain.**

### 1. Based on Functionality and Performance

| Type                                | Description                                                                                                                                                                                                       | Example                                              |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>Stand-alone Embedded Systems</b> | Operate independently without needing a host system. Take input → process → give output.                                                                                                                          | Microwave oven, Washing machine                      |
| <b>Real-time Embedded Systems</b>   | Respond to inputs or events <b>within a strict time constraint</b> . Divided into: • <b>Hard Real-Time:</b> Deadlines must <b>always</b> be met • <b>Soft Real-Time:</b> Occasional deadline misses are tolerable | • Hard: Airbag system • Soft: Video streaming system |
| <b>Networked Embedded Systems</b>   | Connected to a network (wired or wireless) to share data or control other systems.                                                                                                                                | Home automation system, IoT-enabled sensor           |
| <b>Mobile Embedded Systems</b>      | Compact, portable, and power-sensitive systems embedded in mobile devices.                                                                                                                                        | Smartwatch, Smartphone, Digital camera               |

### 2. Based on Performance and Complexity

| Type                                                | Description                                                                | Example                                          |
|-----------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------|
| <b>Small-scale Embedded Systems</b>                 | Use 8-bit or 16-bit microcontrollers; simple software; low cost and power. | Automatic door control, simple sensors           |
| <b>Medium-scale Embedded Systems</b>                | Use 16-bit or 32-bit microcontrollers; more complex hardware/software.     | GPS systems, Industrial control systems          |
| <b>Sophisticated (Large-scale) Embedded Systems</b> | Use powerful processors, RTOS, networking; complex and costly.             | Aircraft avionics, Smart traffic control systems |

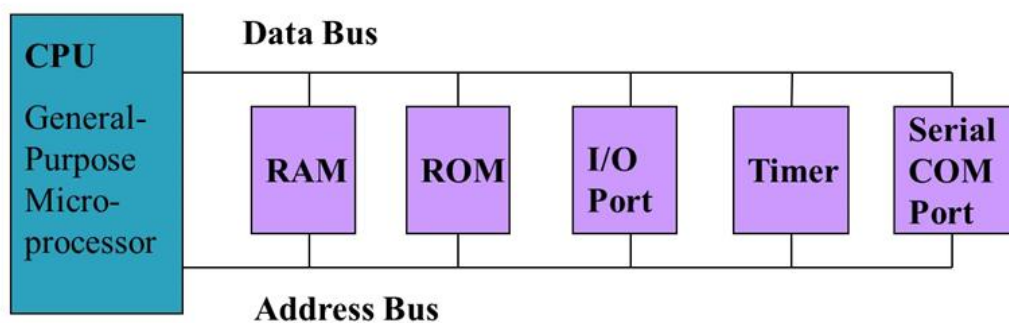
### 3. Based on Real-Time Requirements

| Type                          | Description                                                                      | Example                                   |
|-------------------------------|----------------------------------------------------------------------------------|-------------------------------------------|
| <b>Hard Real-Time Systems</b> | Critical tasks; any delay can cause failure.                                     | Pacemaker, Anti-lock braking system (ABS) |
| <b>Soft Real-Time Systems</b> | Time constraint is important but not critical.                                   | Smart TV, Mobile games                    |
| <b>Firm Real-Time Systems</b> | Occasional deadline misses are allowed, but too many can degrade system quality. | Audio/video conferencing tools            |

#### 4. Based on Application Domain

| Domain               | Example Systems                                             |
|----------------------|-------------------------------------------------------------|
| Consumer Electronics | Washing machines, Smart TVs, Digital cameras                |
| Automotive           | Engine control units (ECUs), Airbag systems, Cruise control |
| Industrial           | Process control systems, Robotics, SCADA                    |
| Medical              | ECG machines, Insulin pumps, MRI machines                   |
| Telecommunication    | Routers, Network switches, Set-top boxes                    |
| Aerospace & Defense  | Radar systems, Missile guidance, Flight control systems     |

#### 6. Features and Architecture Considerations for Embedded Systems:



**CPU/Processor:** It act as a brain to the system, it is used to execute the program and to sending and receiving signals. The processing unit could be a microprocessor, a microcontroller, embedded processor, DSP, ASIC or FPGA selected for an embedded system based on the application requirements.

**ROM for the program:** Nonvolatile (read-only memory, ROM); meaning that it retains its contents even when power is off. It also called Program memory. In embedded systems, the application program after being compiled is saved in the ROM. The processing unit accesses the ROM to fetch instructions sequentially and executes them within the CPU. There are different categories of ROM such as: programmable read only memory (PROM), erasable programmable read only memory (EPROM), electrically erasable programmable read only memory (EEPROM) etc. There is also flash memory which is the updated version of EEPROM and extensively used in embedded systems.

**RAM for data:** Random access memory (RAM) is volatile i.e. it does not retain the contents after the power goes off. It is used as the data memory in an embedded system. It holds the variables declared in the program, the stack and intermediate data or results during program run time. The Processing unit accesses the RAM for instruction execution to save or retrieve data. There are different variations of RAM such as: static RAM (SRAM), dynamic RAM (DRAM), pseudo static RAM (PSRAM), non-volatile RAM (NVRAM), synchronous DRAM, (SDRAM) etc.

**I/O PORTS:** To provide digital communication with the outside world. These Ports are two types Parallel Port and Serial Port, Parallel are used to communicate with Parallel I/O devices like LEDs, LCDs, 7-Segment Displays, Keypads etc., Serial Port is used to communicate with Serial devices like Bluetooth module, Wi-Fi, Zigbee, GSM Modules, PCs etc.

**Address and Data buses:** To link these subsystems to transfer data and instructions. The system bus consists of three different bus systems: address bus, data bus and control bus. Processor sends the address of the destination through the address bus. So address bus is unidirectional from processor to the external end. Data can be sent or received from any unit to any other unit in the diagram. So data bus is bidirectional. Control bus is basically a group of control signals from the processing unit to the external units cases such as creating accurate delays, executing a periodic task, implementing a PWM (Pulse With Modulation) output or capturing the elapsed time between two events, to vary the speed of data transfer rate i.e. Baud rate (bps-bits per second) in case serial communication etc.

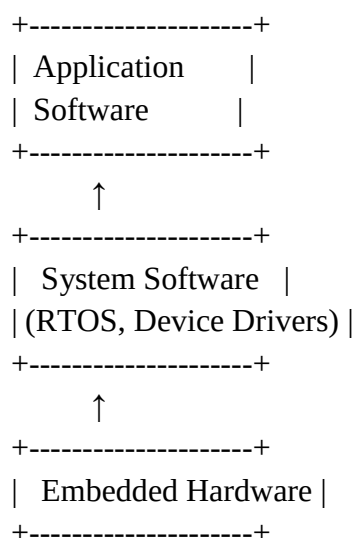
**Clock:** To keep the whole system synchronized. It may be generated internally or obtained from a crystal or external source; modern MCUs offer considerable choice of clocks.

**Watchdog timer:** This is a safety feature, which resets the processor if the program becomes stuck in an infinite loop/hang.

**Communication interfaces:** A wide choice of interfaces is available to exchange information with another IC or System. They include Universal Asynchronous Receiver/Transmitter (UART), Serial Peripheral Interface (SPI), Inter-Integrated Circuit (I<sup>2</sup>C or IIC), Universal Serial Bus (USB), Controller Area Network (CAN), Ethernet, and many others.

**Non-volatile Memory for data:** This is used to store data whose value must be retained even when power is removed. Serial numbers for identification and network addresses are two obvious candidates.

## 7. Architecture of an Embedded System



## Working Flow

1. **Input Device** senses the environment.
2. Input data sent to **Processor**, which reads and processes it using code from **memory**.
3. Processor uses **software logic** and possibly interacts with **communication modules** or **output devices**.
4. **Output Devices** act based on results (display data, control motor, etc.).

## Key Takeaways

- **Processor** is central to the architecture.
- **Software and hardware** tightly coupled for real-time control.
- **RTOS** is used in time-critical systems.
- **Architecture varies** depending on application (simple systems may not use OS).

### 8. General purpose computers vs Embedded system

| Feature                    | General Purpose Computer                       | Embedded System                                         |
|----------------------------|------------------------------------------------|---------------------------------------------------------|
| <b>Purpose</b>             | Designed to perform <b>multiple tasks</b>      | Designed for a <b>specific task</b> or application      |
| <b>Examples</b>            | Laptop, Desktop, Server                        | Microwave, Washing machine, Car ECU                     |
| <b>Flexibility</b>         | Highly flexible and programmable               | Limited flexibility; fixed functions                    |
| <b>User Interface</b>      | Usually has rich UI (keyboard, mouse, monitor) | May have no UI or minimal UI (buttons, LEDs, display)   |
| <b>Hardware Components</b> | Powerful CPU, GPU, RAM, storage, etc.          | Simple CPU or microcontroller, limited RAM and storage  |
| <b>Operating System</b>    | Runs full OS (Windows, Linux, macOS)           | Often runs a <b>Real-Time OS (RTOS)</b> or <b>no OS</b> |
| <b>Power Consumption</b>   | Higher                                         | Lower, optimized for efficiency                         |
| <b>Size</b>                | Larger in size and bulk                        | Compact and integrated                                  |
| <b>Cost</b>                | More expensive                                 | Cost-effective for specific functions                   |
| <b>Performance Focus</b>   | High performance for multitasking              | Real-time and task-specific performance                 |
| <b>Upgradability</b>       | Easily upgradable (software & hardware)        | Usually not upgradable once deployed                    |
| <b>Examples of Tasks</b>   | Web browsing, gaming, video editing            | Controlling motor speed, reading sensor data            |

| Device       | General Purpose or Embedded? | Why?                                                                    |
|--------------|------------------------------|-------------------------------------------------------------------------|
| Laptop       | General Purpose              | Runs many applications like Word, browser, games                        |
| Smartwatch   | Embedded                     | Specific functions like time, fitness tracking                          |
| ATM          | Embedded                     | Designed only to handle banking tasks                                   |
| Mobile Phone | Hybrid                       | Combines general-purpose (apps) with embedded systems (camera, sensors) |

## 9. Embedded System Design Process

Designing an embedded system involves a **systematic approach** to build a product that meets both **functional** and **non-functional** requirements such as performance, power, cost, and reliability.

There are different steps involved in Embedded system design process. These steps depend on the design methodology. Design methodology is important for optimizing performance, and developing computer aided design tools. It also makes communication between team members easier. Figure1 shows the steps in the design process. They are requirements gathering, specification formulation, architecture design, building of components, and system integration.

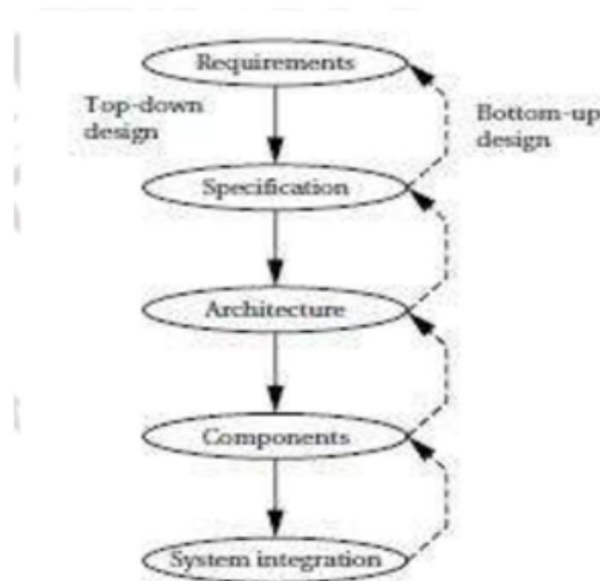


Figure 1. Steps in Design process

The steps in the design process can be viewed as top down view and bottom up view. Top-down view begins with the most abstract description of the system and concludes with concrete details. Bottom-up view starts with components to build a system. Bottom-up design steps are shown in the figure as dashed-line arrows. We need bottom-up design because we do not have perfect insight into how later stages of the design process will turn out.

The major goals of the design to be considered are :

- Manufacturing cost;
- Performance (both overall speed and deadlines) and
- Power consumption.

The tasks which need to be performed at each step are the following.

- We must analyze the design at each step to determine how we can meet the specifications.
- We must then refine the design to add details.

- We must verify the design to ensure that it still meets all system goals, such as cost, speed, and so on.

We will discuss each step of the design process in detail.

Steps in Embedded System Design Process:

## 1. Requirement Analysis

- Understand the **problem** the system is meant to solve.
- Define:
  - Functional requirements (What should the system do?)
  - Non-functional requirements (e.g., cost, power consumption, speed)
  - Real-time constraints (Hard/Soft deadlines)

□ *Example:* A temperature-controlled fan should automatically turn ON/OFF based on room temperature.

## 2. System Specification

- Create detailed specifications for:
  - Hardware (e.g., microcontroller, sensors, interfaces)
  - Software (functions, interfaces, timing)
  - Interfaces with external systems
  - Power and size constraints

□ *Document includes:* Block diagrams, data flow, communication protocols

## 3. System Architecture Design

- Divide the system into **hardware and software components**.
- Choose:
  - Microcontroller vs microprocessor
  - Memory type and size
  - Communication protocols (I2C, UART, etc.)
  - Input/output devices

□ *Goal:* Define how components interact — **architecture block diagram** is created here.

## 4. Hardware Design

- Design and prototype hardware components:
  - Schematic design
  - PCB layout
  - Component selection (sensors, actuators, power supply, etc.)
- Simulate if needed

□ *Tools:* Altium, Eagle, OrCAD

## 5. Software Design & Development

- Write application code based on system requirements
- Use cross-compilation tools for target hardware
- Develop:
  - Device drivers
  - Middleware (if required)
  - Application logic
- Optional: RTOS integration

☐ *Tools:* Keil, MPLAB, Arduino IDE, Embedded C/C++, Python

## 6. Integration and Testing

- Combine hardware and software
- Perform:
  - **Unit Testing** (test individual modules)
  - **Integration Testing** (hardware-software interaction)
  - **System Testing** (overall functionality)
- Debug using serial monitor, logic analyzers, debuggers (e.g., JTAG)

☐ Test against all edge cases and performance constraints

## 7. Validation & Optimization

- Check if the system meets all **user requirements**
- Optimize:
  - Power consumption
  - Response time
  - Memory usage
  - Reliability

☐ Rework design if any bottlenecks or failures occur

## 8. Deployment

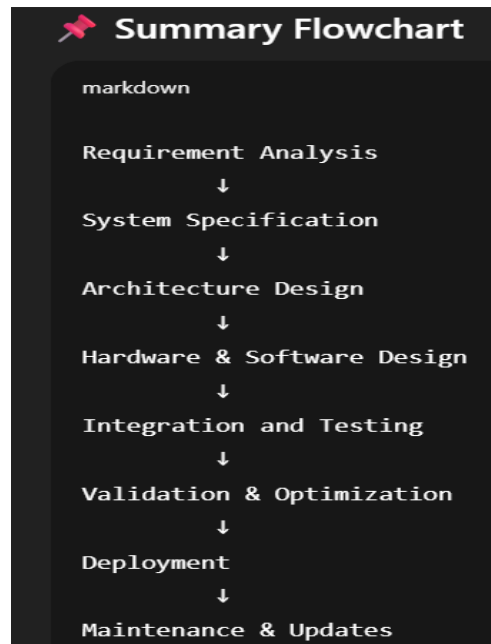
- Finalize and flash code into ROM/Flash
- Prepare the system for real-world use
- Ensure proper packaging and environmental resistance (if needed)

☐ *Deliverable:* Fully functional embedded product

## 9. Maintenance and Updates

- Monitor performance post-deployment
- Fix bugs or release firmware updates
- Provide diagnostics and support tools

☐ Maintenance is easier if remote update capability (OTA) is included.



## 1.1 Requirements

Informal descriptions gathered from the customer are known as requirements. The requirements are refined into a specification to begin the designing of the system architecture. Requirements can be functional or non-functional requirements. Functional requirements need output as a function of input. Non-functional requirements includes performance, cost, physical size, weight, and power consumption. Performance may be a combination of soft performance metrics such as approximate time to perform a user-level function and hard deadlines by which a particular operation must be completed. Cost includes the manufacturing, nonrecurring engineering(NRE) and other costs of designing the system. Physical size and weight are the physical aspects of the final system. These can vary greatly depending upon the application. Power consumption can be specified in the requirements stage in terms of battery life.

### Example:

Let us consider an example of a GPS system.

## 1.2 Specification

Requirements gathered is refined into a specification. Specification serves as the contract between the customers and the architects. Specification is essential to create working systems with a minimum of designer effort. It must be specific, understandable and accurately reflect the customer's requirements.

### Example:

Considering the example of the GPS system, the specification would include details for several components:

- Data received from the GPS satellite constellation
- Map data
- User interface
- Operations that must be performed to satisfy customer requests
- Background actions

### 1.3 Architecture Design

The specification describes only the functions of the system. Implementation of the system is described by the Architecture. The architecture is a plan for the overall structure of the system. It will be used later to design the components. The architecture will be illustrated using block diagrams as shown below.

#### Example:

A basic block diagram of the GPS system shows the major operations and the data flow among the blocks.

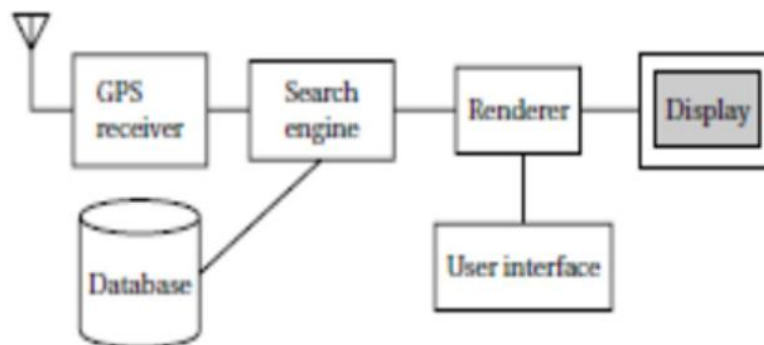


Figure 3. GPS system data flow and operations

This block diagram( figure 3) is an initial architecture that is not based either on hardware or on software but combination of both. This block diagram explains about GPS navigating system where GPS receiver gets current position and the destination is taken from user, digital map for source to destination is found from database and displayed by the renderer. The system block diagram may be refined into two block diagrams – hardware and software.

### 1.3.1 Hardware block diagram :

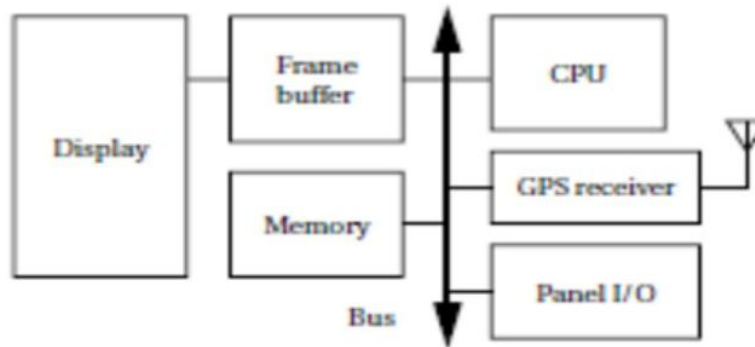


Figure 4. GPS system hardware

Hardware consists of one central CPU surrounded by memory and I/O devices. We have chosen to use two memories that is frame buffer for the pixels to be displayed and separate program/data memory for general use by the CPU. The GPS receiver is used to get the GPS coordinates, and the panel I/O is used to get the destination from the user.

### 1.3.2 Software block diagram :

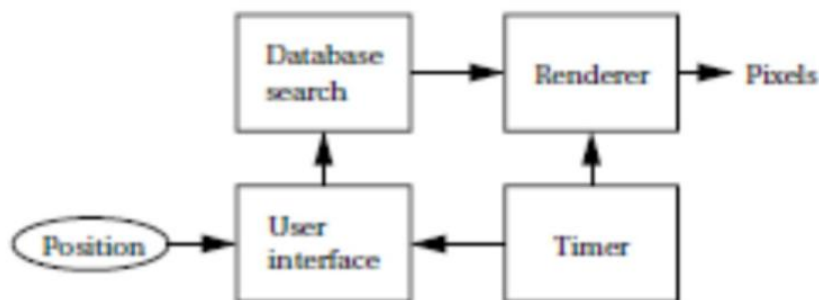


Figure 5. GPS system software

The software block diagram closely follows the system block diagram. We have added a timer to control when we read the buttons on the user interface and render data onto the screen.

To have a truly complete architectural description, we require more details, such as where units in the software block diagram will be executed in the hardware block diagram and when the operations will be performed in time.

Architectural descriptions must be designed to satisfy the functional and non-functional requirements. Not only must all the required functions be present, but we must meet cost, speed, power and other non- functional constraints. Starting out with a system architecture and refining that to hardware and software architectures is one good way to ensure that we meet all specifications. We can concentrate on the functional elements in the system block

diagram, and then consider the non- functional constraints when creating the hardware and software architectures.

How do we know that our hardware and software architectures in fact meet constraints on speed, cost, and so on?

- Estimate the properties of the components in the block diagrams (Example: search and rendering functions in the moving map system)
- Accurate estimation derives in part from experience, both general design and particular experience.
- All the non- functional constraints are estimated. If the decisions are based on bad data, those results will show up only during the final phases of design.

### **1.4 Hardware and Software components**

The architectural description tells us what components we need. The component design effort builds those components in conformance to the architecture and specification. The components in general includes both hardware and software modules. Some of the components will be ready-made (example :CPU, memory chips).

#### **Example:**

In the moving map, GPS receiver is a predesigned standard hardware component. Topographic software is a standard software module which uses standard routines to access the database. Printed circuit board are the components which needs to be designed. Lots of custom programming is required.

When creating these embedded software modules, ensure the system runs properly in real time and that it does not take up more memory space than allowed. The power consumption of the moving map software example is particularly important. You may need to be very careful about how you read and write memory to minimize power. For example, memory transactions must be carefully planned to avoid reading the same data several times, since memory accesses are a major source of power consumption.

### **1.5 System integration**

After the components are built, they are integrated. Bugs are typically found during the system integration. Good planning can help us to find the bugs quickly. By debugging a few modules at a time, simple bugs can be uncovered. By fixing the simple bugs early, more complex or obscure bugs can be uncovered. System integration is difficult because it usually uncovers problems. The debugging facilities for embedded systems are usually much more limited than the desktop systems. Careful attention is needed to insert appropriate debugging facilities during design which can help to ease system integration problems.

## 10. Embedded system with IOT connectivity

An **Embedded System with IoT (Internet of Things) connectivity** is a **smart device** that not only performs dedicated tasks but also **connects to the internet or a network** to share data, receive commands, or coordinate with other systems.

An embedded system with IoT connectivity is an embedded device integrated with **sensors, actuators, processing unit, and communication module**, enabling it to connect to the internet and interact with other devices, systems, or cloud services.

### Key Components:

| Component                        | Description                                                                              |
|----------------------------------|------------------------------------------------------------------------------------------|
| <b>Sensor/Actuator</b>           | To sense environment or perform actions (e.g., temperature sensor, motor)                |
| <b>Microcontroller/Processor</b> | Executes logic and handles data (e.g., ESP32, STM32)                                     |
| <b>Connectivity Module</b>       | Enables communication over internet/network: Wi-Fi, Bluetooth, LoRa, Zigbee, NB-IoT, LTE |
| <b>Power Source</b>              | Battery, mains supply, or solar for portability                                          |
| <b>Cloud or Server</b>           | Data is stored, visualized, or processed on the cloud (e.g., AWS IoT, Google Cloud IoT)  |
| <b>Application Interface</b>     | Mobile apps or web dashboards to monitor/control the system                              |

### Common Connectivity Options:

| Protocol               | Range           | Use Case                       |
|------------------------|-----------------|--------------------------------|
| <b>Wi-Fi</b>           | Medium          | Smart home devices             |
| <b>Bluetooth / BLE</b> | Short           | Wearables, health devices      |
| <b>Zigbee</b>          | Short to medium | Home automation                |
| <b>LoRa</b>            | Long            | Agriculture, remote monitoring |
| <b>NB-IoT / LTE-M</b>  | Long (cellular) | Smart cities, meters           |

### Example Applications:

| Application                     | Description                                                     |
|---------------------------------|-----------------------------------------------------------------|
| <b>Smart Thermostat</b>         | Adjusts temperature based on user settings & weather data       |
| <b>Smart Light System</b>       | Controlled remotely via mobile app or schedule                  |
| <b>Health Monitoring Device</b> | Collects vitals & sends data to cloud for doctors to review     |
| <b>Industrial IoT System</b>    | Monitors machine temperature and vibration, alerts on anomalies |
| <b>Agricultural IoT Node</b>    | Monitors soil moisture and controls irrigation remotely         |

## Working Principle

1. **Sensors** collect data from the environment.
2. Data is processed by the **embedded controller**.

3. Processed data is sent to the **cloud/server** via communication module.
4. User can access this data or control the device via **mobile/web interface**.
5. Device may receive remote commands or software updates.

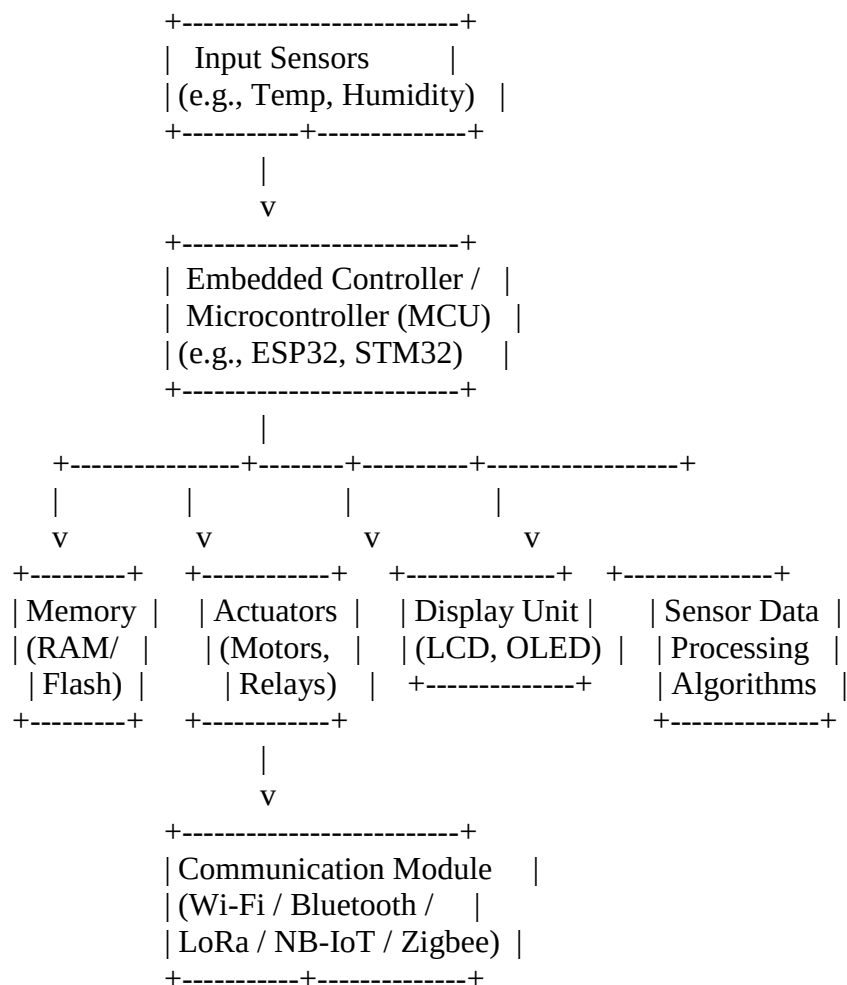
### Example: Smart Water Tank System (with IoT)

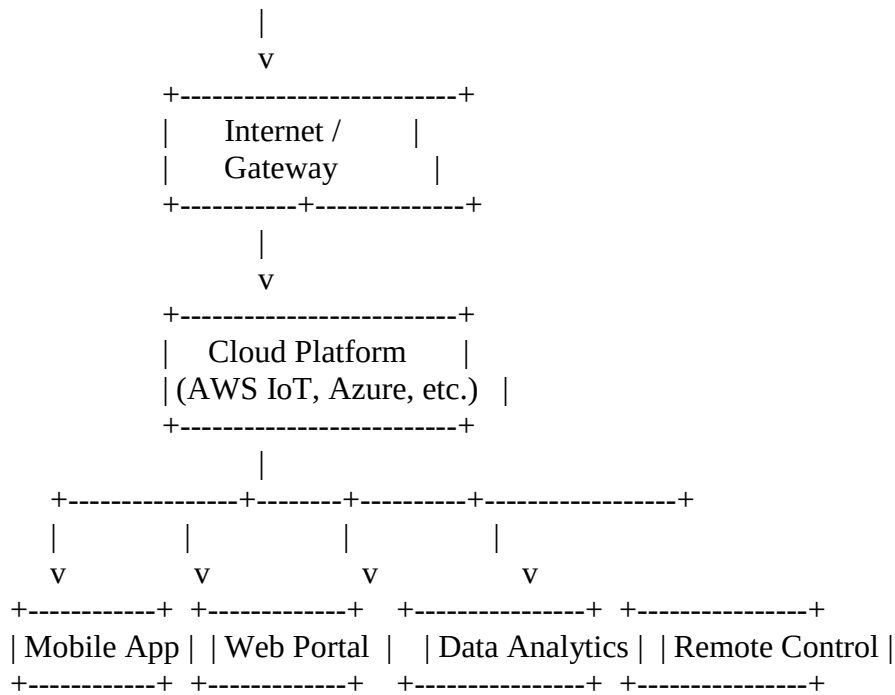
- **Sensor:** Ultrasonic sensor for water level
- **Controller:** ESP32 microcontroller
- **Connectivity:** Wi-Fi
- **Cloud:** Sends data to Thingspeak or Firebase
- **App:** Notifies when water is low or overflows; user can start/stop motor remotely

### Benefits

- Real-time monitoring and control
- Remote access from anywhere
- Predictive maintenance
- Efficient resource utilization
- Data analytics and automation

### Block diagram of a typical IoT-enabled Embedded System:





Here:-

| Block                       | Function                                                      |
|-----------------------------|---------------------------------------------------------------|
| <b>Input Sensors</b>        | Collect real-world data (e.g., temperature, motion, pressure) |
| <b>Embedded Controller</b>  | Executes logic, processes data, manages devices               |
| <b>Actuators</b>            | Perform physical actions (e.g., switch motor on/off)          |
| <b>Communication Module</b> | Sends data to the cloud via internet protocols                |
| <b>Cloud Platform</b>       | Stores data, applies logic, enables control via dashboard     |
| <b>User Interfaces</b>      | Web or mobile apps for user interaction and control           |

### Assignment questions:

1. **Define an embedded system.** How is it different from a general-purpose computer?
2. Explain different **types of memory** used in embedded systems with examples.
3. Compare **microcontroller-based** and **microprocessor-based** embedded systems
4. General purpose computers vs embedded system.
5. Write a short note on **firmware** in embedded systems.